

Name/NetID: Nathan Kariuki / nkari011

Session Date: April 30, 2026 (1-2 hours)

Objective: Add timing, generate the full speedup table, run the lock pool size experiment, and write output files.

Attempts made:

Added chrono timing around each variant. For the lock pool experiment needed a runtime-configurable K value since the template version fixes K at compile time. Created a separate `lock_experiment.cpp` with a `DynLockPool` class that takes K as a constructor argument.

Solutions/Workarounds:

```
auto t0 = Clock::now();
BfsPushNaiveSerial bfs_naive_s(g.num_vertices, 0);
BspSerial(g, bfs_naive_s);
double bfs_naive_s_time = Sec(Clock::now() - t0).count();
cout << "BFS serial naive: " << bfs_naive_s_time << " sec (1x baseline)" << endl;
```

Commits: Add timing, speedup table, lock pool experiment, output files

Experiments/Analysis:

Top 3 in-degrees:

vertex 10029 in-degree=13906

vertex 8737 in-degree=13434

vertex 2914 in-degree=12422

Full speedup table (baseline = serial naive push):

Algorit hm	Serial LF naive	Serial LF dense	Naive parallel 1t	Naive parallel 4t	Dense parallel 1t	Dense parallel 4t	Pull 1t	Pull 4t
BFS	1x	3.71x	0.18x	0.21x	1.36x	1.56x	0.57x	0.68x
SSSP	1x	3.15x	0.27x	0.32x	1.05x	1.20x	1.22x	1.56x
CC	1x	2.11x	0.22x	0.25x	0.50x	0.57x	1.49x	1.89x

Lock pool size experiment:

Algorithm	K = 1024	K = 16384	K = 262144
BFS	1.95s	1.92s	2.59s
SSSP	16.36s	16.28s	19.74s
CC	12.75s	12.39s	15.61s

K = 16384 was fastest for all three. K = 262144 was actually slower than K = 1024 likely because the larger pool causes more cache misses, 262144 mutexes take up significant memory and accessing them across threads thrashes the cache more than the reduced contention saves.

Learning outcome:

Bigger K isn't always better. There's a sweet spot where contention reduction and cache performance balance out. K = 16384 hit that spot on this hardware.